A Survey on Low Level Image Processing Using Kernel Convolutions

Eyad Jawad

Department of Software, College of Computer Science, University of Technology

Statistics and Probability

Dr. Ali Adeel

January 1, 2026

Abstract

We will discuss the different types of kernel convolutions (masks) used in image processing, mainly, blur, and edge detection. We will explain them clearly and implement them ourselves. It would be important to show how the size of the kernel (mask) affects their quality. We will provide examples for the filters we discuss to leave the reader with a better understanding.

Introduction

1.1 Scope

We will try to cover the main subjects of kernel convolutions; Gaussian Blur, Blur, edge detection using Sobel Operator, and then Canny Operator.

1.2 Purpose

In this work, we will try to simplify the ideas as much as possible for the reader, although we will try to mention everything needed to understand the subject, so the reader can easily learn more about the subject. Our goal here is to make the reader knowledgeable enough about how images themselves work, and how image editing and processing takes place.

1.3 Background Information

An image is made of a 2D grid containing tiny cells we call pixels; each pixel has three fundamental values; red, blue, and green, abbreviated as RGB. Each color can have a value between 0 and 255. There are other image formats that are a bit different, in our work, we will not mention how images work, you can assume that an image is a 2D matrix of color values.

BMP is an image format that we will use in this work for its simplicity; it is made of a signature, and a simple 2D color table.

Kernel convolutions are operators whose results solely depend on the surrounding pixels, obtained by performing a convolution between a kernel and a part of the image [6].

A kernel (or sometimes called a mask) is a window that is smaller than the image; it takes the values in it and processes them to output a final number that is our result. A kernel can be as small as 2X2, or as big as 31X31; it depends on the desired result, and the filter type [1]. We will use it when we want to refer to low-level image filters.

1.4 Key Terms: *Gaussian Blur, Basic (normal) Blur, Sobel Operator, Edge detector, Prewitt Operator, mask (kernel), Canny Operator, NMS, Hysteresis Edge Tracking, Double threshold, Gradient.*

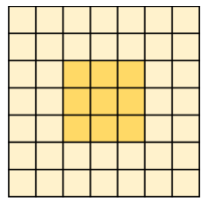


Figure 1. It shows a 7X7 image, and a 3X3 kernel in the middle. This kernel will scan the whole image. The output will be assigned to the center cell.

2. Filters

2.1 Blurs

2.1.1 Basic Blur

Blur is used in modern cameras to enhance certain aspects of an image. It can be simple, like running a mask on an image. However, it can also be quite complex; consisting of many steps involving object recognition AI [2]. Our work here will focus on low level blur only.

Basic blur simply takes the sum of each color and divides by the number of the pixels in the mask (mean). We have noticed that the blur will be more potent relative to the size of the mask. See Figure 2.

A-The original image B-Using 3X3 mask C-Using 9X9 mask D-Using 21X21 mask

Figure 2. Blurring an image using normal blur and different mask sizes

To show how easy it is to implement blur in code, we have written an algorithm to do it. It was designed for learning, so it is not written to implement blur efficiently. We wrote this algorithm to give the reader an idea of how a filter would work, thus we will not write another algorithm in this paper. We will, however, link to a C++ code that has all the kernel convolutions mentioned in this paper.

```
Algorithm: Blur

Input: image A of size Height X Width, MaskSize

Output: blurred image B of size Height X Width


For i = 0 to height - 1 do

        For j = 0 to width - 1 do

                RedColor = 0

                BlueColor = 0

                GreenColor = 0

                Cells = 0

                For ti = -MaskSize/2 to MaskSize/2 do

                        For tj = -MaskSize/2 to MaskSize/2 do

                                Y = i + ti

                                X = j + tj

                                If (y >= 0 and y < Height and x >= 0 and x < width) then

                                        RedColor = RedColor + Image[y][x].red

                                        BlueColor = BlueColor + Image[y][x].blue

                                        GreenColor = GreenColor + Image[y][x].green

                                        Cells = Cells + 1

                                End if

                        End for tj

                End for ti

                BImage[i][j].red = RedColor/Cells

                BImage[i][j].blue = BlueColor/Cells

                BImage[i][j].green = GreenColor/Cells

        End for j

End for i

Print BImage

End
```

2.1.2 Gaussian Blur

This type of blur is much more popular and useful, because it is more useful. It uses the Gaussian –normal distribution (weighted mean) to remove the spikes and high frequency pixels in images, thus eliminating the noise and preserving good edges [3].

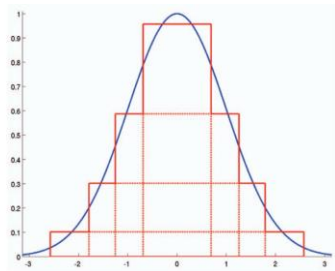


Figure 3. Shows the effect of using the Gaussian blur on 1D values

We can get the Gaussian Blur masks by using the gaussian equation [4]:

$$G(x,\ y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} (1)$$

The variables (x and y) are the distance from the center pixel, and (σ^2) is the variance, increasing its value will increase the blur. The term ($e^{-\frac{x^2+y^2}{2\sigma^2}}$) here means that the effect of the pixels decreases the further the pixel is from the center pixel [6]

We mostly use the discrete approximation of the gaussian kernel (Figure 4); it works quite well. Using the floating-point approximation is about 66 times slower, and more demanding than using the fixed-point approximation [5].

1/16

1	2	1
2	4	2
1	2	1

1/273

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

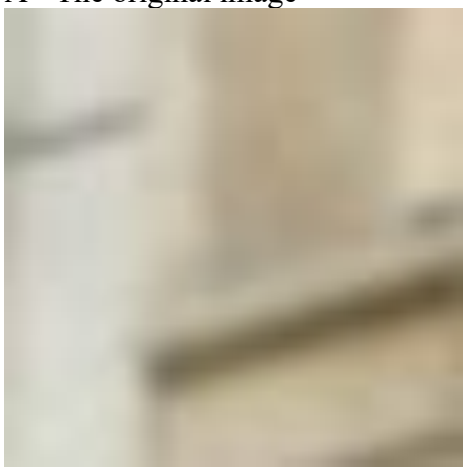
1/1003

0	0	1	2	1	0	0
0	3	13	22	13	3	0
1	13	59	97	59	13	1
2	22	97	159	97	22	2
1	13	59	97	59	13	1
0	3	13	22	13	3	0
0	0	1	2	1	0	0

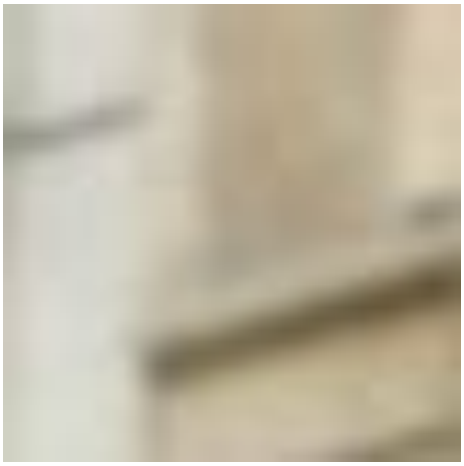
Figure 4. Shows discrete approximation of the Gaussian kernels 3x3, 5x5, 7x7.

A - The original image

B – Applying 3X3 Gaussian Blur



C – Applying 5X5 Gaussian Blur



D – Applying 7X7 Gaussian Blur

Figure 5. It shows one image using different sized Gaussian Blur masks on it.

As it appears in figure 5, the Gaussian Blur smoothed the spikes in pixels very well. Increasing the size of the kernel (mask), however, does not make a noticeable difference, and in many practical examples, using a larger mask does not yield to more accurate results relative to the increase in computational cost.

The applications of the Gaussian Blur in image processing are very important. Even in modern image processing, in removing the noise and softening the images. It also is great at removing the noise at edges which is then used for edge detection filters like the Sobel edge detector. It is also used in image down sampling due to its property in pixel averaging, that is why it is used in image compression and size reduction, and that what makes it so important and wildly used in image processing and photoshop, and it is so easy to implement in code, relative to other approximations and algorithms [6].

2.2 Edge Detection

Edge detectors are filters that work by finding discontinuities in the image. A high difference in brightness or color values means that there is an edge here.

Edge detectors, in general, are used in many fields, such as medical imaging, motion detection, object detection for AI, it is also important for image perception in human, we give more attention to edges, which enhances the quality of an image, and has many more implications that we will not mention here [8] [10].

2.2.1 Sobel Operator

The Sobel operator is one of the simplest edge detectors, to implement and to understand, it takes the surrounding pixels and calculate the difference between the polar pixels (gradient), it does that for the x-axis and the y-axis to check for vertical (G_x) and horizontal (G_y) edges:

-1	0	1
-2	0	2
-1	0	1

x-component

1	2	1
0	0	0
-1	-2	-1

y-component

Figure 6. It shows how the Sobel Operator is calculated for the vertical and horizontal edges.

It then takes, and the final for the center pixel is calculated using this equation [7]:

$$G = \sqrt{G_x^2 + G_y^2}$$

It works because if the contrast between the pixels is high, it means that there is an edge, but if the values of the image colors are quite similar, the result will be 0; no edge. Here, we must pay attention to the values overflowing. In binary, integers and variables in general are allocated a specific number of bits (i.e. memory), which means that there is an upper limit for variables that cannot be exceeded, or the values will overflow, which means that they will wrap around back to zero. In BMP format images, 255 is that upper value, so we must check if they exceed 255; the maximum value for a color in images, otherwise the resulting image will not make sense. Also, we do not actually calculate the values for each color; it will not make a lot of sense. That is why we take the grayed version of the pixels, which we can calculate by taking the mean of the colors for each pixel.

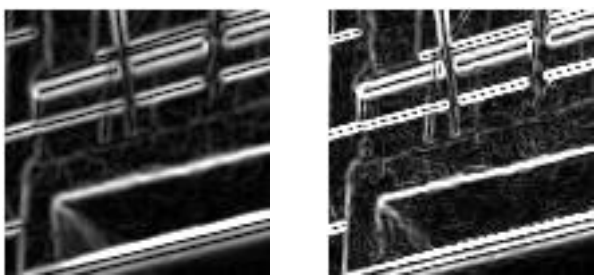
A slight problem that might arise here is that the noise in the finer parts of the image might result in a high contrast which will not make a lot of sense, too. That is why we would

usually do a Gaussian Blur before applying to Sobel Operator, or any other edge detector for that matter, to eliminate noise and spikes that might result in fake edges [7].

A- The original image B- Using the Sobel Operator

Figure 7. It shows the result of the Sobel Operator against the original image

We have mentioned in the Gaussian Blur that one of its uses is smoothing out the pixels for edge detectors, like in the Sobel Operator:



A- Sobel with Gaussian B- Only Sobel

Figure 8. It shows the effects of using the Gaussian Blur before applying the edge detector.

2.2.2 Canny Edge Detector

The Canny edge detector is wieldy used in the field. It uses two adaptive thresholds, based on the mean of the image and gradient magnitude, and standard deviation. Its edges are the most reliable and preserves better edge information [10] [11].

Implementing it is more complex than the Sobel Operator, to implement it we do these five steps:

A. Smoothing: We first use the Gaussian Blur to smooth out the image and eliminate the noise that makes fake edges. The standard 5X5 mask is usually used.

B. Edge Detection: the input of the Canny Edge Detector is a grayscale image that an edge detector was run on, you could use any first order differential detectors like the Sobel Operator, Prewitt Operator, Robert Operator, or two order differential detectors. We will use the Soble Operator.

C. Non-Maximum Suppression (NMS): In this step, we will use the operators of the Sobel Operator to determine the direction of the edge, using that; we will delete edges that are not the maximum in the direction of the edge and only preserve the maximum edges:

$$\theta = \tan^{-1}\left(\frac{Gy}{Gx}\right) \cdot \frac{180}{\pi}$$

We multiply by ($\frac{180}{\pi}$) to change Radian angles to Degree angles for clearer use. After that there are four cases; vertical, horizontal, diagonal1 and diagonal2.

D. Double Threshold: we prompt the user to determine two thresholds; a high one, and a low one. We will remove any edge below the low threshold (no edges) and keep any edge above the high threshold (strong edges).

E. Hysteresis Edge Tracking: for edges between the two thresholds (weak edges); we will check recursively if they are connected to a strong edge in a 3X3 kernel, if they are connected, we will promote them to a strong edge, if they are not connected, we will remove them.

There are many other variations, but this is a general conceptual way to understand the canny edge detector. Using those five steps it will make thin edges one pixel wide. We could not get industry standard results in our implementation, but the concept holds (Figure 9) [11] [10].

There are many other types of edge detectors, such as Prewitt operator, Canny's edge detector, Laplacian of Gaussian, Difference-of-Gaussian, and many more, but we will not talk about them for their complexity, and because we aim for this paper to be as succinct and as simple as we can make it while providing the reader with as much knowledge as possible.

2.3 Other Filters

There are many other types of filters and many of them are for aesthetics only and is not very useful for the industry, but even those could be useful, so we will mention some, and they are mostly not kernel convulsions, thus we will not mention them; they are not the subject of this paper.

A-The original image B-Sobel C- Canny with Gaussian D- Canny without Gaussian

Figure 9. It shows the effect of using the Gaussian Blur on the Canny Operator, and the Canny Operator against the Sobel Operator and the original image.

Conclusion

We talked about blur types; normal blur, which works by taking the mean of the pixels in the mask and assigning it to the center pixel, and the Gaussian Blur, which uses the Gaussian formula to get approximations and is effective in image smoothing, useful in image compression and size reduction, can be extremely useful in eliminating image noise, and lay ground for other filters to function, like edge detectors. We also talked about the Sobel Operator, one of the simplest edge detectors out there, and that it works by finding the difference between the polar pixels, we also talked about how the Canny Operator uses NMS, double threshold and Hysteresis suppression to give us better results than the Sobel Operator, and how useful they are in the of fields medical, engineering, and computer science.

There are many types of low-level filters, some of them are simple, and some of them are not. They can be vital for the industry, such as the Gaussian Blur, or the Canny Operator, and they can be used for aesthetics, like Grayscale, color invert, Vintage, Sepia, Sharpen, and many other filters that we didn't have the chance to talk about in this paper.

We have left a GitHub link for the implementations of these filters for the reader to check on them if they wish to [9].

References

1. Ding, X., Zhang, X., Zhou, Y., Han, J., Ding, G., & Sun, J. (2022). *Scaling up your kernels to 31×31: Revisiting large kernel design in CNNs* (arXiv:2203.06717) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2203.06717

2. Zhang, S., Shen, X., Lin, Z., Měch, R., Costeira, J. P., & Moura, J. M. F. (2018). *Learning to understand image blur*. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 6586–6595). IEEE.

https://openaccess.thecvf.com/content_cvpr_2018/html/Zhang_Learning_to_Understand_CVPR_2018_paper.html

3. Elboher, E., & Werman, M. (2012). *Efficient and accurate Gaussian image filtering using running sums*. In Proceedings of the 2012 12th International Conference on Intelligent Systems Design and Applications (ISDA) (pp. 897–902). IEEE. https://doi.org/10.1109/ISDA.2012.6416657

4. Alhamuy, M., & Erkmen, B. (2023). *Analog image processing using Gaussian based filtering for motion detection applications*. Proceedings of ELECO 2023. https://www.eleco.org.tr/ELECO2023/eleco2023-papers/22.pdf

5. Shipitko, Oleg & Grigoryev, Anton. (2018). *Gaussian filtering for FPGA based image processing with High-Level Synthesis tools.* https://repo.ssau.ru/bitstream/Informacionnye-tehnologii-i-nanotehnologii/Gaussian-filtering-for-FPGA-based-image-processing-with-HighLevel-Synthesis-tools-69075/1/paper_394.pdf

6. Alhamuy, Mecid & Erkmen, Burcu. (2023). *Analog Image Processing Using Gaussian Based Filtering for Motion Detection Applications.* 1-5. 10.1109/ELECO60389.2023.10415976.

7. Sobel, Irwin. (2014). *An Isotropic 3x3 Image Gradient Operator. Presentation at Stanford A.I. Project 1968.*

8. Dawood, Saja. (2023). *A Survey on Various Approaches for Edge Detection. International Journal of Scientific Research in Science, Engineering and Technology.* 27-41. 10.32628/IJSRSET23103142.

9. The GitHub repository that has the code for the filters: https://github.com/Eyad-Jawad/UOT-Image-processing-statistics-project

10. Ahmed, Ahmed. (2018). COMPARATIVE STUDY AMONG SOBEL, PREWITT AND CANNY EDGE DETECTION OPERATORS USED IN IMAGE PROCESSING. Journal of Theoretical and Applied Information Technology. 96. 9.

11. W. Rong, Z. Li, W. Zhang and L. Sun, "*An improved Canny edge detection algorithm*" 2014 IEEE International Conference on Mechatronics and Automation, Tianjin, China, 2014, pp. 577-582, doi: 10.1109/ICMA.2014.6885761.